

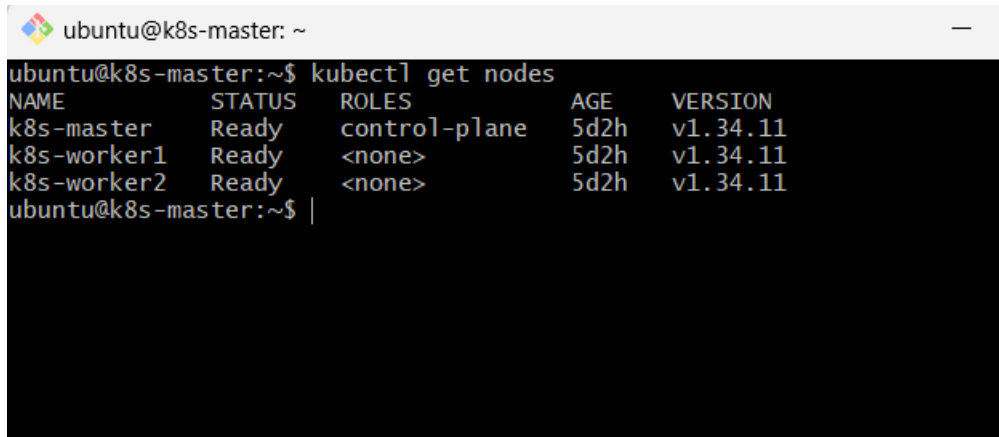
KUBERNETES (K8S) TASK - 05

1. Create a namespace dev-environment and apply a resource-based quota that restricts the number of pods to 3 and services to 2.

Prerequisites

Verify your cluster:

kubectl get nodes

A terminal window titled 'ubuntu@k8s-master: ~' showing the command 'kubectl get nodes' and its output. The output is a table with 5 columns: NAME, STATUS, ROLES, AGE, and VERSION. It lists three nodes: k8s-master (control-plane), k8s-worker1, and k8s-worker2, all in 'Ready' status for 5d2h, running version v1.34.11.

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	Ready	control-plane	5d2h	v1.34.11
k8s-worker1	Ready	<none>	5d2h	v1.34.11
k8s-worker2	Ready	<none>	5d2h	v1.34.11

Create dev-environment namespace and ResourceQuota

Objective

Create a namespace called dev-environment and restrict:

- Maximum Pods = 3
- Maximum Services = 2

Step 1: Create namespace

kubectl create namespace dev-environment

Verify:

kubectl get namespaces

You should see:

dev-environment

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl create namespace dev-environment  
namespace/dev-environment created  
ubuntu@k8s-master:~$ kubectl get namespaces  
NAME                STATUS    AGE  
default             Active    5d2h  
dev-environment     Active    17s  
kube-node-lease     Active    5d2h  
kube-public         Active    5d2h  
kube-system         Active    5d2h  
ubuntu@k8s-master:~$ |
```

Step 2: Create ResourceQuota YAML

Create:

nano dev-resource-quota.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 dev-resource-  
apiVersion: v1  
kind: ResourceQuota  
metadata:  
  name: dev-quota  
  namespace: dev-environment  
spec:  
  hard:  
    pods: "3"  
    services: "2"
```

Step 3: Apply the quota

kubectl apply -f dev-resource-quota.yaml

Expected:

resourcequota/dev-quota created

Step 4: Verify

kubectl get resourcequota -n dev-environment

For detailed information:

kubectl describe resourcequota dev-quota -n dev-environment

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl apply -f dev-resource-quota.yaml
resourcequota/dev-quota created
ubuntu@k8s-master:~$ kubectl get resourcequota -n dev-environment
NAME          REQUEST          LIMIT          AGE
dev-quota     pods: 0/3, services: 0/2    10s
ubuntu@k8s-master:~$ kubectl describe resourcequota dev-quota -n dev-environment
Name:         dev-quota
Namespace:    dev-environment
Resource      Used  Hard
-----
pods          0    3
services      0    2
ubuntu@k8s-master:~$ |

```

Step 5: Test pod limit

Create three pods:

```
kubectl run pod1 --image=nginx -n dev-environment
```

```
kubectl run pod2 --image=nginx -n dev-environment
```

```
kubectl run pod3 --image=nginx -n dev-environment
```

Check:

```
kubectl get pods -n dev-environment
```

Now try a fourth:

```
kubectl run pod4 --image=nginx -n dev-environment
```

It should be rejected because the namespace has a maximum of **3 pods**.

Check quota:

```
kubectl describe resourcequota dev-quota -n dev-environment
```

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl apply -f dev-resource-quota.yaml
resourcequota/dev-quota created
ubuntu@k8s-master:~$ kubectl get resourcequota -n dev-environment
NAME          REQUEST          LIMIT          AGE
dev-quota     pods: 0/3, services: 0/2    10s
ubuntu@k8s-master:~$ kubectl describe resourcequota dev-quota -n dev-environment
Name:         dev-quota
Namespace:    dev-environment
Resource      Used  Hard
-----
pods          0    3
services      0    2
ubuntu@k8s-master:~$ kubectl run pod1 --image=nginx -n dev-environment
kubernetes.io/apiextensions.k8s.io/v1beta1: pod/pod1 created
ubuntu@k8s-master:~$ kubectl run pod2 --image=nginx -n dev-environment
kubernetes.io/apiextensions.k8s.io/v1beta1: pod/pod2 created
ubuntu@k8s-master:~$ kubectl run pod3 --image=nginx -n dev-environment
kubernetes.io/apiextensions.k8s.io/v1beta1: pod/pod3 created
ubuntu@k8s-master:~$ kubectl get pods -n dev-environment
NAME    READY   STATUS    RESTARTS   AGE
pod1    1/1     Running   0          19s
pod2    1/1     Running   0          19s
pod3    1/1     Running   0          19s
ubuntu@k8s-master:~$ kubectl run pod4 --image=nginx -n dev-environment
Error from server (Forbidden): pods "pod4" is forbidden: exceeded quota: dev-quota, requested: pods=1, used: pods=3, limited: pods=3
ubuntu@k8s-master:~$ |

```

2. Create a pod in the prod-environment namespace with 0.2 CPU and 200Mi memory requests, and 0.5 CPU and 500Mi memory limits.

Objective

Create a Pod with:

Resource Request Limit

CPU 0.2 CPU 0.5 CPU

Memory 200Mi 500Mi

In Kubernetes:

0.2 CPU = 200m

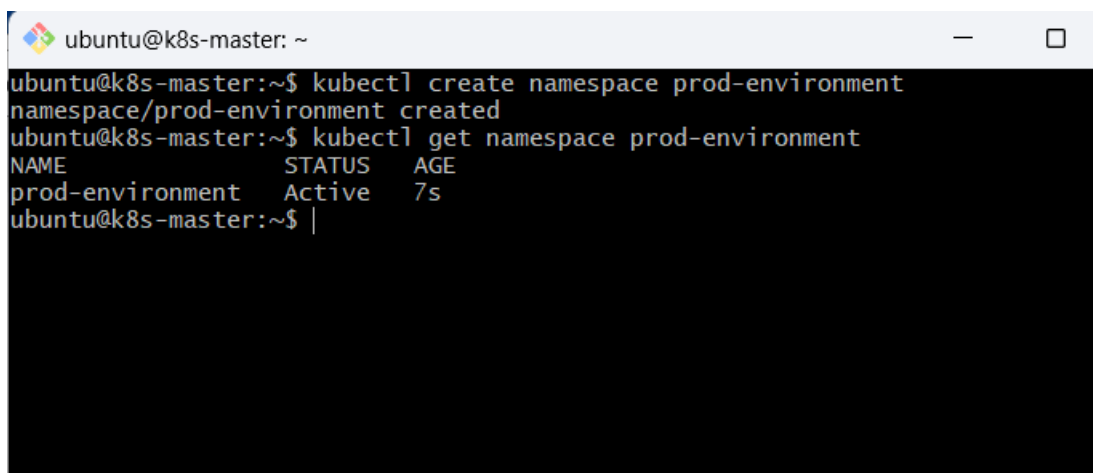
0.5 CPU = 500m

Step 1: Create namespace

`kubectl create namespace prod-environment`

Verify:

`kubectl get namespace prod-environment`

A terminal window titled 'ubuntu@k8s-master: ~' with standard window controls. The terminal shows the following commands and output:

```
ubuntu@k8s-master:~$ kubectl create namespace prod-environment
namespace/prod-environment created
ubuntu@k8s-master:~$ kubectl get namespace prod-environment
NAME                STATUS    AGE
prod-environment    Active   7s
ubuntu@k8s-master:~$ |
```

Step 2: Create Pod YAML

nano prod-pod.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 prod-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: prod-pod  
  namespace: prod-environment  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
    resources:  
      requests:  
        cpu: "200m"  
        memory: "200Mi"  
      limits:  
        cpu: "500m"  
        memory: "500Mi"
```

Step 3: Create the Pod

kubectl apply -f prod-pod.yaml

Expected:

pod/prod-pod created

Step 4: Check Pod

kubectl get pods -n prod-environment

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano prod-pod.yaml  
ubuntu@k8s-master:~$ kubectl apply -f prod-pod.yaml  
pod/prod-pod created  
ubuntu@k8s-master:~$ kubectl get pods -n prod-environment  
NAME      READY   STATUS    RESTARTS   AGE  
prod-pod  1/1     Running   0           9s  
ubuntu@k8s-master:~$ |
```

Step 5: Verify resources

kubectl describe pod prod-pod -n prod-environment

Look for:

Requests:

cpu: 200m

memory: 200Mi

Limits:

cpu: 500m

memory: 500Mi

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl describe pod prod-pod -n prod-environment  
Name: prod-pod  
Namespace: prod-environment  
Priority: 0  
Service Account: default  
Node: k8s-worker1/172.31.16.155  
Start Time: Sun, 27 Sep 2026 12:47:28 +0000  
Labels: <none>  
Annotations: cni.projectcalico.org/containerID: 700e9957daf40611d2f12f7ecc0  
6c401f680ad506469ae7f9869fc016c635b96  
cni.projectcalico.org/podIP: 192.168.194.90/32  
cni.projectcalico.org/podIPs: 192.168.194.90/32  
Status: Running  
IP: 192.168.194.90  
IPs: IP: 192.168.194.90  
Containers:  
  nginx:  
    Container ID: containerd://f9bb0b9754ec985c402adfb7ab8371124ea7d39e6fa15a8  
8731d020f01073f60  
    Image: nginx  
    Image ID: docker.io/library/nginx@sha256:abe47724e466aeab9a345d8e46a22  
1c2fa8953c7848bb4a3bd9976a7199f8cf2  
    Port: <none>  
    Host Port: <none>  
    State: Running  
      Started: Sun, 27 Sep 2026 12:47:29 +0000  
    Ready: True  
    Restart Count: 0  
    Limits:  
      cpu: 500m  
      memory: 500Mi  
    Requests:  
      cpu: 200m  
      memory: 200Mi  
  Environment: <none>  
  Mounts:  
    /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-mk6rk (ro)  
Conditions:  
  Type                               Status  
  PodReadyToStartContainers          True  
  Initialized                         True  
  Ready                              True  
  ContainersReady                    True  
  PodScheduled                       True  
Volumes:  
  kube-api-access-mk6rk:  
    Type: Projected (a volume that contains injected data from multiple sources)  
    TokenExpirationSeconds: 3607  
    ConfigMapName: kube-root-ca.crt  
    Optional: false  
    DownwardAPI: true  
QoS Class: Burstable  
Node-Selectors: <none>  
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
```

3. In the staging-environment namespace, set a LimitRange that assigns default CPU and memory limits (300m CPU, 600Mi memory) and applies a minimum and maximum CPU.

Objective

Create a LimitRange that provides:

Default limits:

CPU = 300m

Memory = 600Mi

Also apply minimum and maximum CPU values.

For this lab, we will use:

Minimum CPU = 100m

Maximum CPU = 1

So:

100m <= CPU <= 1 CPU

Step 1: Create namespace

kubectl create namespace staging-environment

Step 2: Create LimitRange YAML

nano staging-limitrangle.yaml

```
ubuntu@k8s-master:~$ nano staging-limits.yaml
GNU nano 8.7.1 staging-limits.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: staging-limits
  namespace: staging-environment
spec:
  limits:
  - type: Container
    min:
      cpu: "100m"
    max:
      cpu: "1"
    default:
      cpu: "300m"
      memory: "600Mi"
    defaultRequest:
      cpu: "300m"
      memory: "600Mi"
```

What these settings mean

min:

cpu: 100m

A container cannot request less than 100m CPU.

max:

cpu: 1

A container cannot request/limit more than 1 CPU.

default:

cpu: 300m

memory: 600Mi

If the Pod doesn't specify limits, Kubernetes automatically assigns these limits.

defaultRequest:

cpu: 300m

memory: 600Mi

If the Pod doesn't specify requests, Kubernetes automatically assigns these requests.

Step 3: Apply LimitRange

```
kubectl apply -f staging-limitrange.yaml
```

Step 4: Verify

kubectl get limitrange -n staging-environment

Detailed:

kubectl describe limitrange staging-limits -n staging-environment

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f staging-limitrange.yaml  
limitrange/staging-limits created  
ubuntu@k8s-master:~$ kubectl get limitrange -n staging-environment  
NAME          CREATED AT  
staging-limits 2026-09-27T12:53:11Z  
ubuntu@k8s-master:~$ kubectl describe limitrange staging-limits -n staging-environment  
Name:         staging-limits  
Namespace:    staging-environment  
Type          Resource  Min  Max  Default Request  Default Limit  Max Limit/Request Ratio  
-----  
Container     cpu        100m  1    300m             300m           -  
Container     memory     -     -    600Mi           600Mi          -  
ubuntu@k8s-master:~$
```

Step 5: Create Pod without resources

Create:

nano staging-pod.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 staging-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: staging-pod  
  namespace: staging-environment  
spec:  
  containers:  
  - name: nginx  
    image: nginx
```

Check:

kubectl get pod staging-pod -n staging-environment

Now verify:

kubectl describe pod staging-pod -n staging-environment

You should see Kubernetes automatically assigned:

Requests:

cpu: 300m

memory: 600Mi

Limits:

cpu: 300m

memory: 600Mi

This demonstrates how a **LimitRange** automatically supplies default resource values.

```
ubuntu@k8s-master: ~$ kubectl apply -f staging-pod.yaml
pod/staging-pod created
ubuntu@k8s-master: ~$ kubectl get pod staging-pod -n staging-environment
NAME          READY   STATUS    RESTARTS   AGE
staging-pod   1/1     Running   0           9s
ubuntu@k8s-master: ~$ kubectl describe pod staging-pod -n staging-environment
Name:          staging-pod
Namespace:     staging-environment
Priority:       0
Service Account: default
Node:          k8s-worker2/172.31.26.51
Start Time:    Sun, 27 Sep 2026 12:59:04 +0000
Labels:        <none>
Annotations:   cnf.projectcalico.org/containerID: b79c83d8996c0dc54536810c406f0d9ce11e8bd4f38e9116222d815f77a0c72a
               cnf.projectcalico.org/podIP: 192.168.126.23/32
               cnf.projectcalico.org/podIPs: 192.168.126.23/32
               kubernetes.io/limit-ranger: LimitRanger plugin set: cpu, memory request for container nginx; cpu, memory limit for container nginx
Status:        Running
IP:            192.168.126.23
IPs:           192.168.126.23
Containers:
  nginx:
    Container ID:   containerd://02a55e37e375f92a46e764ba5cc845c1f05445aa5106ebf2e43a3cd31cbd5ab5
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:abe47724e466aeab9a345d8e46a221c2fa8953c7848bb4a3bd9976a7199f8cf2
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Sun, 27 Sep 2026 12:59:05 +0000
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:          300m
      memory:       600Mi
    Requests:
      cpu:          300m
      memory:       600Mi
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-wgvp6 (ro)
Conditions:
  Type                               Status
  PodReadyToStartContainers         True
  Initialized                        True
  Ready                             True
  ContainersReady                   True
  PodScheduled                      True
Volumes:
  kube-api-access-wgvp6:
    Type:          Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:  kube-root-ca.crt
    Optional:      false
    DownwardAPI:   true
QoS Class:         Guaranteed
Node-Selectors:    <none>
Tolerations:       node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
```

4. Create a pod and a NodePort service in the default namespace, then create another pod in the test namespace and communicate between them using Service DNS.

Objective

We will:

1. Create a Pod in default
2. Create a NodePort Service for it
3. Create another Pod in test
4. Communicate from the test Pod to the service using Kubernetes DNS.

The important concept is:

test namespace Pod

|

| DNS

v

nginx-service.default.svc.cluster.local

|

v

nginx Pod

Step 1: Create test namespace

kubectl create namespace test

Verify:

kubectl get namespace test

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl create namespace test  
namespace/test created  
ubuntu@k8s-master:~$ kubectl get namespace test  
NAME      STATUS    AGE  
test      Active    7s  
ubuntu@k8s-master:~$ |
```

Step 2: Create nginx Pod in default namespace

Create:

nano nginx-default.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 nginx  
apiVersion: v1  
kind: Pod  
metadata:  
  name: nginx-default  
  labels:  
    app: nginx-default  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
    ports:  
    - containerPort: 80
```

Apply:

```
kubectl apply -f nginx-default.yaml
```

Verify:

```
kubectl get pods
```

You should see:

```
nginx-default 1/1 Running
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f nginx-default.yaml  
pod/nginx-default created  
ubuntu@k8s-master:~$ kubectl get pods  
NAME             READY   STATUS    RESTARTS   AGE  
nginx-default    1/1     Running   0           7s
```

Step 3: Create NodePort Service

Create:

```
nano nginx-nodeport.yaml
```

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 nginx  
apiVersion: v1  
kind: Service  
metadata:  
  name: nginx-service  
  namespace: default  
spec:  
  type: NodePort  
  selector:  
    app: nginx-default  
  ports:  
  - port: 80  
    targetPort: 80  
    nodePort: 30085
```

Apply:

```
kubectl apply -f nginx-nodeport.yaml
```

Expected:

service/nginx-service created

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano nginx-nodeport.yaml  
ubuntu@k8s-master:~$ kubectl apply -f nginx-nodeport.yaml  
The Service "nginx-service" is invalid: spec.ports[0].nodePort: Invalid value: 30080: provided port is already allocated  
ubuntu@k8s-master:~$ nano nginx-nodeport.yaml  
ubuntu@k8s-master:~$ kubectl apply -f nginx-nodeport.yaml  
service/nginx-service created  
ubuntu@k8s-master:~$
```

tep 4: Verify Service

```
kubectl get svc
```

Expected similar to:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
nginx-service	NodePort	10.x.x.x	<none>	80:30080/TCP

Check endpoints:

```
kubectl get endpoints nginx-service
```

You should see the Pod IP.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get svc  
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)  
AGE  
apache-service                      ClusterIP           10.98.222.38     <none>           80/TCP  
3d6h  
firstpod-service                    ClusterIP           10.98.115.180    <none>           80/TCP  
4d3h  
kubernetes                          ClusterIP           10.96.0.1        <none>           443/TCP  
5d3h  
nginx-loadbalance                   NodePort            10.103.212.154   <none>           80:30081/TCP  
3d5h  
nginx-nodeport-service              NodePort            10.103.57.154    <none>           80:30080/TCP  
3d5h  
nginx-service                       NodePort            10.96.102.166    <none>           80:30085/TCP  
63s  
ubuntu@k8s-master:~$
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get endpoints nginx-service  
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 Endpoints  
lice  
NAME            ENDPOINTS          AGE  
nginx-service    192.168.194.91:80  2m7s  
ubuntu@k8s-master:~$
```

Step 5: Create test Pod

We need a Pod that has a tool such as curl.

Create:

nano test-pod.yaml

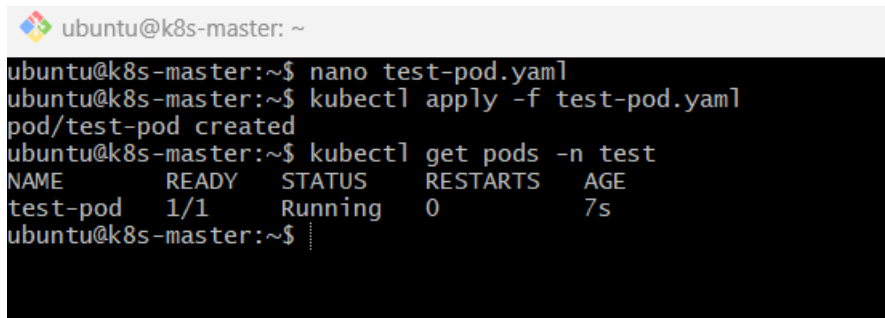
```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 test-pod.yaml *  
apiVersion: v1  
kind: Pod  
metadata:  
  name: test-pod  
  namespace: test  
spec:  
  containers:  
  - name: curl  
    image: curlimages/curl  
    command:  
    - sleep  
    - "3600"
```

Apply:

```
kubectl apply -f test-pod.yaml
```

Check:

```
kubectl get pods -n test
```

A terminal window with a light gray title bar showing 'ubuntu@k8s-master: ~'. The terminal output is as follows:

```
ubuntu@k8s-master:~$ nano test-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f test-pod.yaml
pod/test-pod created
ubuntu@k8s-master:~$ kubectl get pods -n test
NAME      READY   STATUS    RESTARTS   AGE
test-pod  1/1     Running   0           7s
ubuntu@k8s-master:~$
```

Step 6: Test Service DNS

Enter the test Pod:

```
kubectl exec -it test-pod -n test -- sh
```

Inside the container, run:

```
curl http://nginx-service.default.svc.cluster.local
```

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl exec -it test-pod -n test -- sh
~ $ curl http://nginx-service.default.svc.cluster.local
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
~ $ curl http://nginx-service.default.svc.cluster.local:80
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>

```

Important

The **NodePort** is mainly for accessing the Service from outside the cluster:

Node IP:30080

Inside the Kubernetes cluster, Pods normally communicate through the **Service DNS name and Service port 80**, not through the NodePort.

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

5. Apply a LimitRange with a max limit/request ratio of 2 for memory in the performance-environment namespace, and test by creating a pod with mismatched resource requests and limits.

LimitRange with Memory Limit/Request Ratio = 2

Objective

Create a LimitRange in:

performance-environment

with:

maxLimitRequestRatio:

memory: 2

This means:

Memory Limit / Memory Request $\leq$ 2

For example:

Valid

Request = 200Mi

Limit = 400Mi

$400 / 200 = 2$

Valid.

Invalid

Request = 200Mi

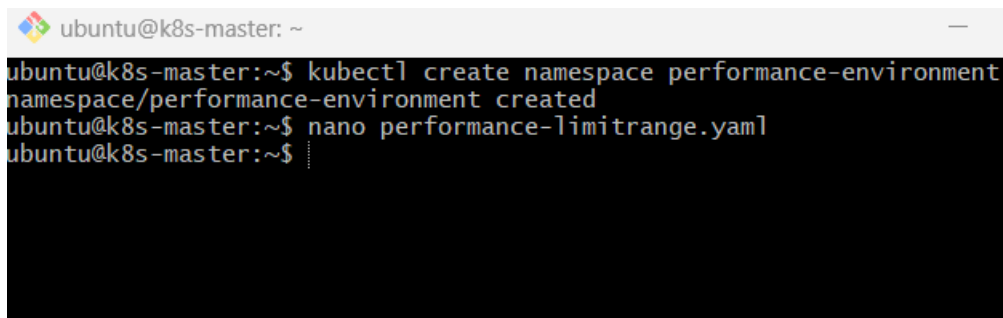
Limit = 500Mi

$500 / 200 = 2.5$

Rejected.

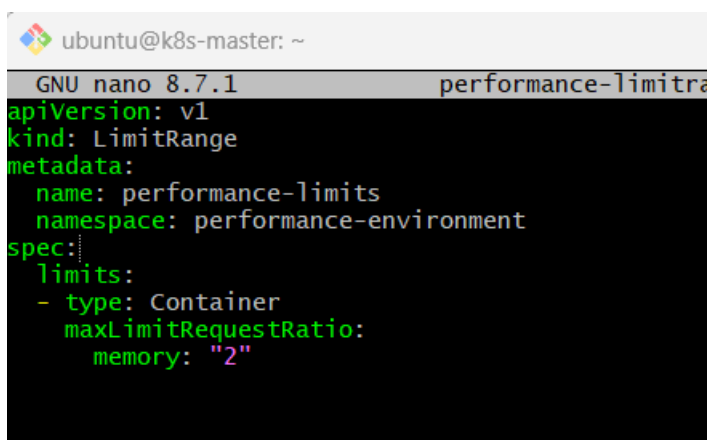
Step 1: Create namespace

kubectl create namespace performance-environment

A terminal window with a light gray title bar showing 'ubuntu@k8s-master: ~'. The terminal output shows the command 'kubectl create namespace performance-environment' being executed, resulting in 'namespace/performance-environment created'. Below this, the command 'nano performance-limitrange.yaml' is entered, and the prompt returns to 'ubuntu@k8s-master:~\$'.

Step 2: Create LimitRange

nano performance-limitrange.yaml

A terminal window showing the GNU nano 8.7.1 editor. The title bar shows 'ubuntu@k8s-master: ~' and the file name 'performance-limitrange.yaml'. The content of the file is a YAML configuration for a LimitRange. The configuration includes 'apiVersion: v1', 'kind: LimitRange', and 'metadata' with 'name: performance-limits' and 'namespace: performance-environment'. The 'spec' section contains a 'limits' list with one entry: 'type: Container' and 'maxLimitRequestRatio: memory: "2"'. The text is color-coded: green for keywords and red for strings.

Step 3: Apply

kubectl apply -f performance-limitrange.yaml

Expected:

limitrange/performance-limits created

Verify:

kubectl get limitrange -n performance-environment

Detailed:

kubectl describe limitrange performance-limits -n performance-environment

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f performance-limitrange.yaml  
limitrange/performance-limits created  
ubuntu@k8s-master:~$ kubectl get limitrange -n performance-environment  
NAME                               CREATED AT  
performance-limits                 2026-09-27T13:19:19Z  
ubuntu@k8s-master:~$ kubectl describe limitrange performance-limits -n performance-environment  
Name:                             performance-limits  
Namespace:                         performance-environment  
Type:                               

| Type      | Resource | Min | Max | Default | Request | Default Limit | Max Limit/Request Ratio |
|-----------|----------|-----|-----|---------|---------|---------------|-------------------------|
| Container | memory   | -   | -   | -       | -       | -             | 2                       |


```

Step 4: Create a valid Pod

Let's first test a valid configuration.

Create:

nano performance-valid-pod.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 performance-valid-po  
apiVersion: v1  
kind: Pod  
metadata:  
  name: performance-valid  
  namespace: performance-environment  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
    resources:  
      requests:  
        memory: "200Mi"  
      limits:  
        memory: "400Mi"
```

Apply:

```
kubectl apply -f performance-valid-pod.yaml
```

Expected:

pod/performance-valid created

Check:

```
kubectl get pod -n performance-environment
```

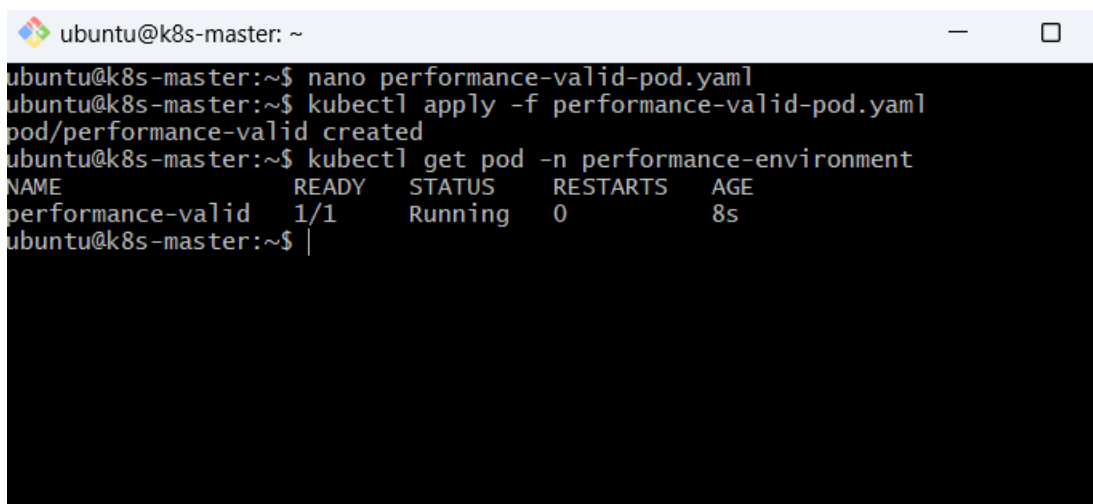
It should be:

performance-valid 1/1 Running

Because:

$400\text{Mi} / 200\text{Mi} = 2$

which satisfies the maximum ratio.

A terminal window titled 'ubuntu@k8s-master: ~' with standard window controls. The terminal shows the following commands and output:

```
ubuntu@k8s-master:~$ nano performance-valid-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f performance-valid-pod.yaml
pod/performance-valid created
ubuntu@k8s-master:~$ kubectl get pod -n performance-environment
```

NAME	READY	STATUS	RESTARTS	AGE
performance-valid	1/1	Running	0	8s

```
ubuntu@k8s-master:~$ |
```

Step 5: Create an invalid Pod

Now intentionally create a mismatched request/limit.

```
nano performance-invalid-pod.yaml
```

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 performance-invalid-pod  
apiVersion: v1  
kind: Pod  
metadata:  
  name: performance-invalid  
  namespace: performance-environment  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
    resources:  
      requests:  
        memory: "200Mi"  
      limits:  
        memory: "500Mi"
```

Apply:

kubectl apply -f performance-invalid-pod.yaml

This should be rejected.

Why?

Memory request = 200Mi

Memory limit = 500Mi

$$500 / 200 = 2.5$$

But our LimitRange allows:

Maximum ratio = 2

Therefore:

$$2.5 > 2$$

and Kubernetes rejects the Pod.

You should see an error similar to:

Error from server (Forbidden)

with information indicating that the memory limit/request ratio exceeds the allowed value.

 ubuntu@k8s-master: ~

```
ubuntu@k8s-master:~$ nano performance-invalid-pod.yaml
```

```
ubuntu@k8s-master:~$ kubectl apply -f performance-invalid-pod.yaml
```

```
Error from server (Forbidden): error when creating "performance-invalid-pod.yaml": pods "performance-invalid" is forbidden: memory max limit to request ratio per Container is 2, but provided ratio is 2.500000
```

```
ubuntu@k8s-master:~$
```